

Cypher 5 vs Cypher 25 : Comparaison sur des Données de Vols Américains

Projet de Bases de Données Spécialisées - Master

Introduction

Ce projet compare les performances et l'expressivité de Cypher 5 et Cypher 25 sur un dataset réel de vols. L'objectif est de vérifier empiriquement les problèmes de complexité identifiés dans la littérature académique (notamment l'article SIGMOD sur les dangers du list processing) et de tester les solutions proposées par la version 25 du langage.

Le dataset choisi représente les vols américains de la première semaine de janvier 2015 : 107 230 vols entre 313 aéroports, opérés par 14 compagnies. Cette structure de graphe est idéale pour tester des algorithmes de chemins, avec des propriétés numériques sur les arêtes (distance, retard) permettant des requêtes complexes.

Choix et Modélisation des Données

Source des données

Dataset Kaggle “2015 Flight Delays and Cancellations” (US Department of Transportation) :

- **Source** : 5,8 millions de vols sur l'année 2015
- **Échantillon retenu** : Première semaine de janvier (1-7 janvier)
- **Raison** : Volume gérable tout en conservant une densité suffisante pour observer des phénomènes intéressants

Nettoyage des données

Le script `scripts/normalize_data.py` effectue plusieurs transformations :

1. **Filtrage temporel** : Sélection des vols du 1er au 7 janvier 2015
2. **Gestion des timestamps** :
 - Conversion des horaires HHMM vers ISO 8601
 - Traitement du minuit (2400 -> 0000)
 - Détection des vols de nuit (arrivée le lendemain)
3. **Suppression des valeurs manquantes** : Retrait des vols sans horaires de départ/arrivée
4. **Filtrage des aéroports** : Seuls les aéroports utilisés dans les vols sont conservés

Résultat : Réduction de 323 à 313 aéroports, 107 230 vols exploitables.

Modèle de graphe Neo4j

Nœuds :

- Airport (313) : Code IATA, nom, ville, état, coordonnées GPS
- Airline (14) : Code IATA, nom de la compagnie

Relations :

- FLIGHT (107 230) : Propriétés :
 - `departure_ts` / `arrival_ts` : Timestamps ISO 8601
 - `distance` : Distance en miles
 - `delay` : Retard au départ en minutes (peut être négatif)
 - `airline` : Code de la compagnie

Contraintes :

- Unicité des codes IATA (airports et airlines)
- Index sur `departure_ts`, `distance`, `delay` pour optimiser les requêtes

Choix de modélisation : Chaque vol est une relation unique, identifiée par son timestamp. Cela évite le problème des multi-edges : ANC->SEA à 23h54 et ANC->SEA à 08h30 sont deux relations distinctes, ce qui simplifie les requêtes de chemins temporels.

Modèle relationnel PostgreSQL

airlines (14 rows)

- `iata_code` (PK)
- `name`

airports (313 rows)

- `iata_code` (PK)
- `name`, `city`, `state`, `country`
- `latitude`, `longitude`

flights (107 230 rows)

- `id` (PK, SERIAL)
- `source` (FK -> `airports.iata_code`)
- `target` (FK -> `airports.iata_code`)
- `airline` (FK -> `airlines.iata_code`)
- `departure_ts`, `arrival_ts`
- `distance`, `delay`
- CHECK (`source` != `target` AND `distance` > 0)

Index sur `source`, `target`, `departure_ts` pour optimiser les requêtes récursives.

Comparaison des Requêtes

1. Chemins avec propriété croissante

Problème : Trouver des itinéraires LAX->JFK où le retard augmente à chaque escale.

Cette requête illustre le problème central de l'article SIGMOD : l'utilisation de `reduce()` dans une clause `WHERE` rend la requête NP-complète.

Cypher 5 (NOT EXISTS) :

```
MATCH path = (start:Airport {iata_code: 'LAX'})
  -[:FLIGHT*2..4]->(end:Airport {iata_code: 'JFK'})
WHERE NOT EXISTS {
  WITH path
  UNWIND range(0, size(relationships(path))-2) AS i
  WITH relationships(path) AS rels, i
  WHERE rels[i].delay >= rels[i+1].delay
  RETURN 1
}
RETURN [n IN nodes(path) | n.iata_code] AS route
LIMIT 50;
```

Stratégie : Génère TOUS les chemins (319 631), puis filtre *a posteriori*.

Résultats :

- Temps : ~1,5 seconde
- DB Hits : 16 340 381
- Rows traitées : 320 799
- Résultats finaux : 52 chemins valides

Cypher 25 (allReduce) :

```
MATCH path = (start:Airport {iata_code: 'LAX'})
  ((:Airport)-[:FLIGHT]->(:Airport)){2,4}
  (end:Airport {iata_code: 'JFK'})
WHERE allReduce(
  prev_delay = -999999.0,
  rel IN f |
  CASE
    WHEN rel.delay > prev_delay THEN rel.delay
    ELSE null
  END,
  prev_delay IS NOT NULL
)
RETURN path LIMIT 50;
```

Stratégie : Filtre PENDANT la traversée grâce à l'opérateur `Repeat(Trail)`.

Résultats :

- Temps : ~3,5 ms
- DB Hits : 39 904
- Rows traitées : 9 290
- Résultats finaux : 50 chemins valides

Gain : 409x moins d'accès DB, 415x plus rapide. La différence s'accroîtrait avec un graphe plus dense.

Plan d'exécution :

- Cypher 5 : `VarLengthExpand` génère tout puis `Apply+Anti` filtre
- Cypher 25 : `Repeat(Into,Trail)` avec pruning inline

2. Quantified Graph Patterns

Problème : Trouver des itinéraires avec exactement N escales.

Cypher 25 introduit la syntaxe `{n,m}` pour exprimer des répétitions de patterns. C'est un sucre syntaxique qui réduit drastiquement la verbosité.

Cypher 5 (explicite) :

```
MATCH path = (start:Airport {iata_code: 'LAX'})
  -[:FLIGHT]->(hub1:Airport)
  -[:FLIGHT]->(hub2:Airport)
  -[:FLIGHT]->(end:Airport {iata_code: 'JFK'})
WHERE start <> hub1 AND start <> hub2 AND start <> end
  AND hub1 <> hub2 AND hub1 <> end
  AND hub2 <> end
RETURN [n IN nodes(path) | n.iata_code] AS route
LIMIT 10;
```

7 lignes, 6 comparaisons pour éviter les cycles. Si on veut 2 OU 3 escales, il faut dupliquer la requête avec UNION.

Cypher 25 (quantified) :

```
MATCH path = (start:Airport {iata_code: 'LAX'})
  ((-[:FLIGHT]->(:Airport)){3}
  (end:Airport {iata_code: 'JFK'}))
WHERE allReduce(
  seen = [],
  n IN nodes(path) | seen + n,
  single(x IN seen WHERE x = n)
)
RETURN [n IN nodes(path) | n.iata_code] AS route
LIMIT 10;
```

1 ligne pour le pattern, `allReduce()` pour détecter les cycles. Pour un range (2-3 escales), on change juste `{3}` en `{2,3}`.

Réduction :

- 86% de code en moins pour un nombre fixe
- 50% de requêtes en moins pour un range (évite UNION)

Intérêt : Améliore la maintenabilité. Modifier la profondeur = changer un chiffre vs réécrire le pattern entier. Réduit les risques d'oublier une clause WHERE.

3. Plus courts chemins pondérés

Problème : Trouver l'itinéraire JFK->DAY minimisant la distance totale.

C'est ici que les limites de Cypher pur deviennent flagrantes.

Cypher 5 (`shortestPath`) :

```
MATCH path = shortestPath(
  (start:Airport {iata_code: 'JFK'})-[:FLIGHT*]->(end:Airport {iata_code: 'DAY'})
)
RETURN [n in nodes(path) | n.iata_code] AS route,
       reduce(dist = 0, r in relationships(path) | dist + r.distance) AS total_distance;
```

Résultat :

- Route : [JFK, ATL, DAY]
- Distance : 1192 miles
- Temps : 15 ms

Problème : `shortestPath()` minimise le nombre de sauts, pas la distance. Le vrai chemin optimal (JFK->BWI->DAY) fait 590 miles mais passe par 2 sauts aussi. L'algorithme BFS ne considère pas les poids.

Cypher 25 (exploration exhaustive) :

```
MATCH path = (start:Airport {iata_code: 'JFK'})-[:FLIGHT*1..2]->(end:Airport {iata_code: 'DAY'})
WITH path, reduce(dist = 0, r in relationships(path) | dist + r.distance) AS total_distance
ORDER BY total_distance
LIMIT 1
RETURN [n in nodes(path) | n.iata_code] AS route, total_distance;
```

Résultat :

- Route : [JFK, BWI, DAY]
- Distance : 590 miles (optimal)
- Temps : 293 ms

Explosion combinatoire :

- *1..2 (2 sauts max) : 293 ms

- *1..3 (3 sauts max) : 137 secondes (facteur 467x!)

Au-delà de 2-3 sauts, la requête devient inutilisable. Cypher n'a pas de priority queue pour implémenter Dijkstra efficacement.

GDS Dijkstra :

```
CALL gds.graph.project('flights-weighted', 'Airport', {
  FLIGHT: { properties: 'distance' }
});

MATCH (source:Airport {iata_code: 'JFK'})
MATCH (target:Airport {iata_code: 'DAY'})
CALL gds.shortestPath.dijkstra.stream('flights-weighted', {
  sourceNode: source,
  targetNode: target,
  relationshipWeightProperty: 'distance'
})
YIELD totalCost, nodeIds
RETURN [nodeId in nodeIds | gds.util.asNode(nodeId).iata_code] AS route,
       totalCost AS total_distance;
```

Résultat :

- Route : [JFK, BWI, DAY]
- Distance : 590 miles (optimal garanti)
- Temps : 37 ms

Comparaison :

Approche	Temps	Distance	Optimal	Complexité
Cypher <code>shortestPath</code>	15 ms	1192 mi	Non	BFS (min sauts)
Cypher exhaustif (d<=2)	293 ms	590 mi	Oui	O(branches ^{depth})
Cypher exhaustif (d<=3)	137 s	590 mi	Oui	Exponentiel
GDS Dijkstra	37 ms	590 mi	Oui	O(E log V)

Conclusion : Pour des chemins pondérés, GDS est obligatoire. Cypher pur n'a pas les structures de données nécessaires (priority queue).

4. Implémentation d'algorithmes GDS en Cypher 25

Objectif : Reproduire des algorithmes de GDS avec du Cypher pur pour comparer les approches.

Degree Centrality :

GDS :

```
CALL gds.degree.stream('my-graph')
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).iata_code AS airport, score
ORDER BY score DESC LIMIT 10;
```

Cypher 25 :

```
MATCH (a:Airport)
OPTIONAL MATCH (a)-[:FLIGHT]->()
WITH a, count(*) AS out_degree
OPTIONAL MATCH (a)<-[:FLIGHT]-()
WITH a, out_degree, count(*) AS in_degree
RETURN a.iata_code AS airport, out_degree + in_degree AS degree
ORDER BY degree DESC LIMIT 10;
```

Résultat : Performances similaires pour ce cas simple (algorithme linéaire).
Cypher 25 est plus lisible et ne nécessite pas de projection.

Triangle Count :

GDS :

```
CALL gds.triangleCount.stream('my-graph')
YIELD nodeId, triangleCount
RETURN gds.util.asNode(nodeId).iata_code AS airport, triangleCount
ORDER BY triangleCount DESC LIMIT 10;
```

Cypher 25 :

```
MATCH (a:Airport)-[:FLIGHT]->(b:Airport)-[:FLIGHT]->(c:Airport)-[:FLIGHT]->(a)
RETURN a.iata_code AS airport, count(*) AS triangles
ORDER BY triangles DESC LIMIT 10;
```

Problème : Cette requête compte chaque triangle 3 fois (une fois par sommet).
Correction :

```
WITH a, count(*) / 3 AS triangleCount
```

Observation : Pour des algorithmes polynomiaux simples, Cypher 25 est compétitif. L'overhead de GDS (projection du graphe) peut même le rendre plus lent.

Limite : Dès qu'on passe à des algorithmes nécessitant des structures de données avancées (Dijkstra, PageRank, Betweenness Centrality), GDS devient indispensable. Ces algorithmes ne sont pas implémentables efficacement en Cypher.

Équivalents SQL

Chemins avec propriété croissante

SQL ne supporte pas nativement les path queries. Il faut utiliser une requête récursive :

```

WITH RECURSIVE paths AS (
  -- Cas de base : vols directs depuis LAX
  SELECT
    source,
    target,
    ARRAY[source, target] AS route,
    ARRAY[delay] AS delays,
    1 AS hops,
    delay AS last_delay
  FROM flights
  WHERE source = 'LAX'

  UNION ALL

  -- Cas récursif : ajouter un vol
  SELECT
    f.source,
    f.target,
    p.route || f.target,
    p.delays || f.delay,
    p.hops + 1,
    f.delay
  FROM paths p
  JOIN flights f ON p.target = f.source
  WHERE f.delay > p.last_delay
    AND p.hops < 4
    AND NOT (f.target = ANY(p.route))
)
SELECT route, delays
FROM paths
WHERE target = 'JFK' AND hops >= 2
LIMIT 50;

```

Différences :

- Plus verbeux (25 lignes vs 15 pour Cypher 5, vs 10 pour Cypher 25)
- Nécessite de gérer manuellement les cycles (`NOT (f.target = ANY(p.route))`)
- Moins lisible : la logique métier (delay croissant) est noyée dans la syntaxe récursive

Performance : Comparable à Cypher 5 (~1-2 secondes). SQL n'a pas d'équivalent à `allReduce()` pour optimiser.

Avantage : Moins de risque d'écrire accidentellement une requête NP-complète. La verbosité de SQL pousse le développeur à réfléchir.

Plus courts chemins

SQL récursif avec BFS :

```
WITH RECURSIVE paths AS (
  SELECT
    source,
    target,
    ARRAY[source, target] AS route,
    distance,
    1 AS hops
  FROM flights
  WHERE source = 'JFK'

  UNION ALL

  SELECT
    f.source,
    f.target,
    p.route || f.target,
    p.distance + f.distance,
    p.hops + 1
  FROM paths p
  JOIN flights f ON p.target = f.source
  WHERE p.hops < 3
    AND NOT (f.target = ANY(p.route))
)
SELECT route, distance
FROM paths
WHERE target = 'DAY'
ORDER BY distance
LIMIT 1;
```

Problème : Même limitation que Cypher pur. Sans priority queue, impossible d'implémenter Dijkstra efficacement. La requête explore exhaustivement.

Performance : Comparable à Cypher exhaustif (centaines de ms pour 2 sauts, timeout à 3+).

Analyse des Plans d'Exécution

Cypher 5 : VarLengthExpand + Apply + Anti

Exemple du plan pour la requête “delays croissants” :

Operator	Rows	DB Hits
ProduceResults	50	0

Projection	50	1 300	
Limit	50	0	
Apply	52	0	
Anti	52	0	
Limit	319578	0	
Filter	319578	1283196	
VarLengthExpand	319631	15055885	
MultiNodeIndexSeek	0	0	
+-----+-----+-----+			

Observation :

1. VarLengthExpand génère 319 631 chemins (15M DB hits)
2. Pour CHAQUE chemin, Apply lance un sous-plan qui :
 - Unwind les relations du chemin
 - Vérifie si au moins une paire viole la contrainte croissante
 - Si oui, le chemin est rejeté (Anti)
3. Résultat : 52 chemins valides, mais après avoir traité 320k rows

Problème : Le filtre agit trop tard. Le moteur ne peut pas optimiser car le prédicat est dans une sous-requête corrélée.

Cypher 25 : Repeat(Trail) avec pruning

+-----+-----+-----+			
Operator	Rows	DB Hits	
+-----+-----+-----+			
ProduceResults	50	0	
Projection	50	1 004	
Limit	50	0	
NullifyMetadata	50	0	
Repeat(Into, Trail)	50	0	
Filter	6362	12724	
Projection	9289	15651	
Expand(All)	9290	9295	
MultiNodeIndexSeek	1	4	
+-----+-----+-----+			

Observation :

1. Repeat(Trail) intègre directement le filtre allReduce()
2. À chaque expansion (Expand), la projection calcule prev_delay
3. Le filtre rejette immédiatement les chemins invalides (9289 vers 6362)
4. Résultat : Seulement 6362 chemins explorés au total

Avantage : Le pruning s'effectue pendant la traversée. Le moteur évite d'explorer des branches inutiles.

Différence clé :

- Cypher 5 : Generate puis Filter (trop tard)
- Cypher 25 : Generate + Filter (en ligne)

PostgreSQL : Recursive CTE

QUERY PLAN

```

-----
Limit
-> Sort
    -> CTE Scan on paths
        -> Recursive Union
            -> Seq Scan on flights
            -> Hash Join
                -> CTE Scan on paths
                -> Hash (Seq Scan on flights)

```

Observation :

- PostgreSQL explore aussi exhaustivement (pas de pruning avancé)
- Le filtre `f.delay > p.last_delay` s'applique dans le JOIN
- Performance similaire à Cypher 5

Différence : SQL optimise moins bien les path queries car ce n'est pas son cas d'usage principal. Les CTE récursives sont moins optimisées que les opérateurs natifs de Neo4j.

Limitations et Extensions Possibles

Limites du projet

1. **Période courte :** 1 semaine de données limite la profondeur des chemins testables (max 2-3 escales réalistes dans notre graphe).
2. **Pas de contraintes temporelles strictes :** On n'a pas implémenté de requêtes vérifiant que l'arrivée d'un vol précède le départ du suivant (avec marge pour correspondance). C'est faisable avec `allReduce()` mais complexifie les requêtes.
3. **GDS vs SQL :** Pas de comparaison GDS vs requêtes PostgreSQL optimisées (ex: pgRouting pour Dijkstra). Aurait pu enrichir l'analyse.

Extensions intéressantes

Chemins temporellement valides :

```

WHERE allReduce(
  prev_arrival = datetime('1970-01-01T00:00:00'),
  rel IN flights |
  CASE
    WHEN duration.between(prev_arrival, rel.departure_ts).minutes >= 60

```

```

        THEN rel.arrival_ts
        ELSE null
    END,
    prev_arrival IS NOT NULL
)

```

Filtrerait les itinéraires avec minimum 1h de correspondance.

Comparaison avec d'autres SGBD :

- **MemGraph** : Implémente Cypher 25, serait intéressant pour benchmarker
- **DuckDB** : Implémente SQL/PGQ (Property Graph Queries), nouveau standard ISO qui ressemble à Cypher

Algorithmes GDS avancés :

- PageRank : Identifier les hubs (aéroports centraux)
- Betweenness Centrality : Aéroports critiques (plus court chemin passe souvent par eux)
- Community Detection : Groupes d'aéroports fortement connectés (régions géographiques)

Conclusion

Ce projet confirme empiriquement les résultats théoriques de l'article SIGMOD :

1. **Cypher 5 avec `reduce()` dans `WHERE` ne scale pas** : 16M DB hits vs 40k pour la même requête en Cypher 25 (facteur 409x).
2. **`allReduce()` résout le problème** : En intégrant le filtre dans la traversée, Cypher 25 évite l'explosion combinatoire.
3. **Quantified patterns simplifient le code** : Réduction de 50-86% de code pour des patterns répétitifs. Améliore la maintenabilité.
4. **GDS est indispensable pour les chemins pondérés** : Cypher pur (même v25) n'a pas les structures de données pour Dijkstra. L'exploration exhaustive timeout au-delà de 2-3 sauts.
5. **SQL n'est pas adapté aux path queries** : Plus verbeux, moins lisible, performances comparables à Cypher 5 (pas d'équivalent à `allReduce()`).

Recommandations :

- Utiliser Cypher 25 pour toutes les requêtes de chemins avec contraintes
- Privilégier GDS pour algorithmes complexes (Dijkstra, PageRank, etc.)
- Réserver SQL aux requêtes relationnelles classiques (agrégations, jointures simples)

Perspective : L'évolution de Cypher montre que les langages de requêtes graphes mûrent. La standardisation GQL (ISO 2024) bénéficiera de ces leçons. Les quantified patterns et `allReduce()` devraient faire partie de la spécification pour éviter les pièges de Cypher 5.